

AURORA

Workflows et Architecture Multi-Agents

Documentation Technique de Référence



Logiciel d'Intelligence Tactique

February 22, 2026

1 Architecture et Workflows des Agents Aurora

1.1 Vue d'ensemble du processus

Le workflow d'ingestion et de nettoyage automatise le passage des données brutes vers une intelligence exploitable via l'écosystème **Aurora** :

- ⚙️ **Extraction** : Récupération de la télémétrie via l'API Riot Games.
- ⚙️ **Transformation** : Nettoyage et normalisation des structures JSON.
- ⚙️ **Validation** : Audit par LLM pour garantir l'intégrité des données.
- ⚙️ **Persistence** : Stockage unifié en SQLite et CSV.

1.2 Rôle du CleanerBot

Agent : CleanerBot

Le **CleanerBot** agit comme l'agent d'ingénierie des données. Ses responsabilités principales incluent :

- ✅ **Ingestion** : Récupération des données télémétriques brutes via l'API Riot.
- ✅ **Validation** : Vérification de l'intégrité et de la conformité des schémas.
- ✅ **Normalisation** : Mapping unifié pour une analyse cohérente entre agents.
- ✅ **Préparation** : Agrégation des métriques pour les diagnostics tactiques.
- ✅ **Stockage** : Persistence sécurisée dans l'entrepôt de données Aurora.

1.3 Flux de travail détaillé (Ingestion)

Le diagramme de séquence pour l'analyse des performances des joueurs illustre clairement le rôle du CleanerBot :

1. **Requête d'analyse** : Un utilisateur (Joueur/Analyste) envoie une requête d'analyse de performance au Système API.
2. **Récupération et nettoyage** : Le Système API demande au CleanerBot de récupérer et nettoyer les données de match.
3. **Accès à l'API Riot Games** : Le CleanerBot récupère les données de match brutes auprès de l'API de Riot Games.
4. **Stockage des données nettoyées** : Le CleanerBot stocke les données nettoyées dans l'entrepôt de données (Data Warehouse).
5. **Fourniture du jeu de données** : Le CleanerBot fournit le jeu de données nettoyé à l'AnalystBot pour traitement ultérieur.

1.4 AnalystBot (Analyse Tactique)

L'**AnalystBot** est l'agent analyste tactique, intervenant après le **CleanerBot** pour transformer les données nettoyées en informations exploitables.

Ses responsabilités principales sont les suivantes :

Responsabilités de l'AnalystBot

- ⚙️ **Métrique KPI** : Calcul du KDA, de l'Aim et du score propriétaire **OVR**.
- ⚙️ **Audit Tactique** : Identification des failles de timing et de placement.
- ⚙️ **Analyse SQL** : Extraction de caractéristiques de performance ciblées.

Flux de travail de l'AnalystBot (Exemple : Analyse de Performance Joueur)

1. Le **CleanerBot** fournit un jeu de données nettoyé à l'AnalystBot.
2. L'AnalystBot interroge l'entrepôt de données (Data Warehouse) pour obtenir les métriques de performance.
3. Il calcule les KPI et les insights.
4. Ces informations sont ensuite transmises à l'API du système pour être affichées à l'utilisateur.

1.5 CoachBot (IA Stratégique et RAG)

Le **CoachBot** est l'agent d'IA stratégique, spécialisé dans la recommandation de tactiques et la synthèse d'analyses.

Son rôle est crucial pour fournir des conseils exploitables :

Actions du CoachBot

- 💡 **Synthèse** : Génération de rapports naturels et méta-contextuels.
- 💡 **Stratégie** : Recommandations tactiques basées sur la méta actuelle.
- 💡 **Mémoire RAG** : Utilisation de FAISS pour intégrer le jeu professionnel.

Flux de travail du CoachBot (Exemple : Recommandation de Stratégie)

1. L'AnalystBot fournit les faiblesses et le contexte au CoachBot.
2. Le CoachBot interroge la mémoire RAG (FAISS) pour récupérer les stratégies pertinentes.
3. Il collabore avec l'OracleBot pour évaluer la probabilité de succès de la stratégie.
4. Enfin, il envoie les recommandations stratégiques à l'API du système.

1.6 OracleBot (Agent de Prédiction)

L'**OracleBot** est l'agent de prédiction, dont la fonction principale est de prévoir les résultats et d'évaluer les probabilités de succès des stratégies :

 **Prédiction** : Prévission des probabilités de victoire via modèles ML.

 **Évaluation** : Simulation du taux de succès des stratégies proposées.

Flux de travail de l'OracleBot (Exemple : Prédiction du Résultat de Match)

1. L'AnalystBot extrait les caractéristiques prédictives et les envoie à l'OracleBot.
2. L'OracleBot exécute son modèle de prédiction.
3. Il renvoie le result prédit et la probabilité à l'API du système.

1.7 QueryTunerBot (Agent d'Optimisation SQL)

Le **QueryTunerBot** est un agent d'optimisation autonome, conçu pour améliorer les performances des requêtes SQL analytiques :

 **Optimisation** : Restructuration autonome des requêtes DuckDB.

 **Performance** : Simplification active des jointures et filtres.

Flux de travail du QueryTunerBot

1. Une requête SQL est envoyée au QueryTunerBot.
2. Le QueryTunerBot analyse la requête, détecte les goulots d'étranglement (en utilisant `EXPLAIN ANALYZE` de DuckDB).
3. Il réécrit la requête et recommande des index ou des vues matérialisées.
4. La requête SQL optimisée est ensuite exécutée sur DuckDB.

1.8 Orchestration Globale (LangChain)

L'ensemble de ces agents est orchestré par **LangChain**, qui définit les workflows précis et les dépendances entre eux.

Cette approche garantit une collaboration structurée et une séparation claire des préoccupations, où chaque agent apporte son expertise spécifique pour atteindre l'objectif global de la plateforme .